PORTFOLIO PROJECT DOCUMENTATION



Expense Tracker

Frontend-only Personal Finance Dashboard

React + Vite	Recharts	localStorage	Responsive UI

Created by

Ann Noor Muhammad Mrinmoy

Complete technical explanation, learning guide, project motive, architecture, logic, runbook and viva preparation

SECTION 00

How to Use This Guide

A portfolio-ready document and a beginner-friendly learning reference.

This document explains the final Expense Tracker project from two perspectives: first as a professional software project, and second as a learning guide that explains why each technical decision exists.

> **সহজভাবে কীভাবে পড়বে**
>
> প্রথমে Project Overview, Motivation এবং Architecture পড়ো। তারপর Transaction, Financial Logic, Budget, Analytics এবং localStorage অংশ পড়লে পুরো app-এর data flow পরিষ্কার হবে। Viva-এর আগে শেষের Quick Revision এবং Viva Q&A section দেখলেই মূল conceptগুলো দ্রুত revise করা যাবে।

Portfolio View Professional overview, features, architecture and limitations that a reviewer can scan quickly.	**Learning View** Bangla explanations clarify how React state, calculations, storage and validation work.
Runbook Exact npm commands and project structure make the application easy to open in VS Code.	**Viva Preparation** Short Q&A and formula summaries help explain the project confidently.

Source basis: this guide was prepared from the final project archive and its actual source files, including package.json, App.jsx, AppContext.jsx, pages, components, utilities, README.md and the final audit notes.

SECTION 01

Contents

The document follows the same mental model used to design and understand the application.

02	**Project Overview & Executive Summary**
03	**Project Motive & Idea Origin**
04	**Problem, Goals & Scope**
05	**Feature Map**
06	**Technology Stack & Why Each Tool Exists**
07	**Application Architecture**
08	**Folder Structure & Responsibilities**
09	**State Management & Data Flow**
10	Transaction Data Model & CRUD
11	Validation & Data Integrity
12	Financial Calculation Logic
13	Monthly, Category & Trend Logic
14	Budget System & Alerts
15	Analytics & Charts
16	Search, Filters & Sorting
17	Recurring Templates
18	CSV, JSON Backup & Restore
19	localStorage Persistence
20	Reports & Printing
21	Theme, Currency & Responsive UI
22	Accessibility & UX Engineering
23	How the Project Was Built
24	How to Run & Build
25	Testing & Final Audit
26	What I Learned
27	Limitations & Privacy

28	Future Improvements
29	Portfolio Presentation Notes
30	Viva Q&A
31	Quick Revision Cheat Sheet

SECTION 02

Project Overview & Executive Summary

A modern, offline-capable personal finance dashboard built entirely in the browser.

Expense Tracker is a single-page personal finance application designed to help users record income and expenses, understand their current balance, monitor monthly spending, set budgets, view analytics, manage recurring transaction templates, export data and create printable reports. The application is intentionally frontend-only: it does not require a backend server, user account, cloud database or paid API.

The project goes beyond a basic CRUD tutorial. It combines transaction management with derived financial calculations, data validation, browser persistence, chart preparation, reporting, accessibility, responsive design and backup/restore workflows. This makes it suitable as a portfolio project because it demonstrates both user-interface work and real application logic.

Application Type	Frontend-only Single Page Application (SPA)
Primary Purpose	Personal income, expense, budget and financial-summary tracking
Data Storage	Browser localStorage using versioned JSON data
Frontend Framework	React 19.1.1
Build Tool	Vite 6.4.3
Routing	React Router DOM 7.8.1
Charts	Recharts 2.15.4
Icons	Lucide React 0.468.0
Default Currency	BDT / ৳, with USD, EUR, GBP and INR display options
Backend / Database	None

> বাংলায় এক লাইনে **project**টা কী?
> এটা এমন একটি personal finance dashboard যেখানে user income/expense লিখে রাখে, app সেই data থেকে balance, savings, budget status, category spending এবং charts calculate করে; সব data browser-এর localStorage-এ থাকে।

SECTION 03

Project Motive & Idea Origin

Why this project exists and what problem-solving skills it demonstrates.

The core idea starts from a simple everyday problem: people often know that they are spending money, but they do not have a clear, visual answer to questions such as “How much do I have?”, “Where did my money go?”, “Am I within budget?” and “How is this month different from the previous month?”

A useful portfolio version of an expense tracker should therefore do more than store rows. It should convert raw transactions into meaningful information, stay easy to run, and demonstrate practical frontend engineering without requiring deployment of a backend service.

> **Project motive / উদ্দেশ্য**
>
> এই project-এর motive ছিল একটি simple কিন্তু real-world problem কে complete frontend application-এ convert করা। শুধু Add/Delete button নয়; data থেকে insight বের করা, budget manage করা, chart দেখানো, offline persistence রাখা, backup নেওয়া এবং report print করা—এই সব মিলিয়ে practical software engineering practice করা।

- Make personal spending easier to understand through a dashboard instead of raw notes or spreadsheets.
- Demonstrate React state management, component architecture and utility-driven business logic.
- Show that a useful app can be built without a database when the scope is personal/local usage.
- Practice safe data persistence, validation, import/export and edge-case handling.
- Create a project that looks credible in a portfolio and can be explained in a university viva.



Concept evolution: problem -> scope -> data model -> logic -> UI -> persistence -> audit.

SECTION 04

Problem, Goals & Scope

Clear boundaries are what keep the project technically honest.

Problem Statement

Users need a simple way to record financial activity and immediately understand balance, monthly cash flow, spending categories and budget usage without creating an account or sending personal transaction data to a server.

Primary Goals

- Reliable transaction CRUD: add, edit and delete income/expense entries.
- Useful financial summaries: all-time balance, monthly income, expense, savings and savings rate.
- Practical planning: overall monthly budgets and category-specific budgets.
- Visual understanding: income-vs-expense, category distribution and monthly trend charts.
- Data ownership: local persistence, CSV export, JSON backup and printable monthly reports.
- Good UX: light/dark theme, responsive layouts, accessible modals, confirmations and feedback.

Explicit Non-Goals

- No bank integration, payment processing or live exchange-rate conversion.
- No server-side authentication or multi-user accounts.
- No automatic cloud synchronization across devices.
- No background scheduler while the browser is closed.
- No claim of production-grade financial security for sensitive real-world data.

> কেন **scope clear** রাখা জরুরি?
>
> Frontend-only project-এ যদি background recurring payment, secure cloud account বা live currency conversion claim করা হতো, সেটা technically misleading হতো। এই project যেটা সত্যি করতে পারে, শুধু সেটাই feature হিসেবে দেখায়।

SECTION 05

Feature Map

The application is organized around five user-facing areas plus shared settings.

Dashboard Balance, selected-period income/expense/savings, budget progress, insight, trends and recent activity.	**Transactions** Search, month/type/category/payment filters, date range, sort, add/edit/delete, CSV import/export.
Analytics Income vs expense, spending distribution, monthly trend, top categories and largest expenses.	**Budget & Recurring** Overall budget, category limits, status/alerts and weekly/monthly/yearly recurring templates.
Reports Month-specific financial report with totals, budget status and transaction table for A4 printing.	**Settings** Currency display, theme, category management, demo data, JSON backup/restore and reset.

A strong design decision is that the same transaction state feeds all these features. When one transaction changes, the dashboard, analytics, budget totals and reports all derive their new values from the updated state instead of maintaining separate copies of financial totals.

SECTION 06

Technology Stack & Why Each Tool Exists

Every dependency has a clear responsibility.

HTML	The Vite index.html provides the root document and mount point for React.
JavaScript	Core programming language for application logic, calculations, validation and browser APIs.
JSX	Lets React components describe UI using JavaScript-friendly markup.
React 19.1.1	Component-based interface, state-driven updates and reusable UI composition.
React DOM 19.1.1	Renders the React application into the browser DOM.
Vite 6.4.3	Development server and production bundler with fast startup and modern module handling.
React Router DOM 7.8.1	Maps URLs such as /transactions, /analytics and /reports to pages.
Recharts 2.15.4	Builds the dashboard and analytics charts from derived transaction data.
Lucide React 0.468.0	Consistent SVG icons for navigation, actions and status indicators.
Modern CSS	Theme variables, cards, responsive grids, mobile layouts, print rules and visual hierarchy.
localStorage	Persists JSON data in the current browser without a backend.
Blob / File APIs	Create downloadable CSV/JSON files and read user-selected imports in the browser.

কেন **React?**

এই app-এ একই data change হলে অনেক জায়গা update হয়—balance card, chart, budget, recent transactions, report ইত্যাদি। React-এর state-driven rendering এই dependency management সহজ করে এবং UI কে reusable component-এ ভাগ করতে দেয়।

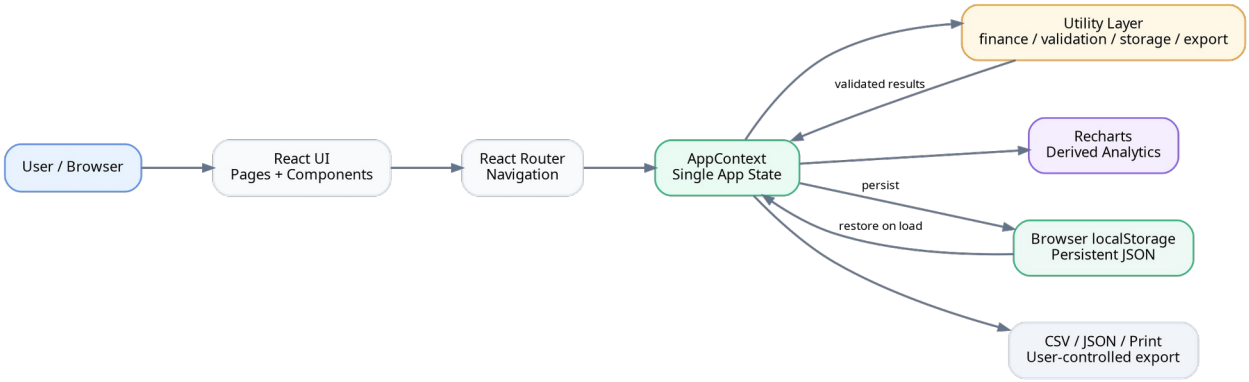
কেন **Vite?**

Vite project setup, dev server, ES module handling এবং production bundling সহজ করে। npm run dev দিয়ে development server এবং npm run build দিয়ে optimized production bundle তৈরি হয়।

SECTION 07

Application Architecture

A layered frontend architecture keeps UI, state and business logic separated.



High-level architecture of the final Expense Tracker project.

The user interacts with React pages and components. React Router controls which page is visible. Shared application state and actions live in AppContext. Calculation, validation, storage, data-integrity and export responsibilities are delegated to utility modules. localStorage persists the state between browser sessions, while Recharts and report components consume derived values rather than storing their own financial truth.

> **Architecture** সহজভাবে
>
> UI নিজে balance calculate করে না। UI AppContext থেকে transactions নেয়; finance utility ওই transactions দিয়ে balance/summary বানায়; state change হলে React আবার render করে। এই separation-এর কারণে একই formula বিভিন্ন component-এ আলাদা আলাদা copy করতে হয় না।

Presentation Layer	pages/ + components/ - what users see and interact with
State / Action Layer	context/AppContext.jsx - shared state, CRUD actions, confirmations, alerts
Business Logic Layer	utils/finance.js + utils/validation.js
Integrity Layer	utils/dataIntegrity.js - sanitizes stored/imported data shapes
Persistence Layer	utils/storage.js - safe localStorage access and data versioning
Export Layer	utils/exportUtils.js - CSV/JSON file generation and CSV parsing

SECTION 08

Folder Structure & Responsibilities

The codebase is compact but intentionally separated by responsibility.

Final project structure

```
expense-tracker-localstorage/
├ package.json / vite.config.js / index.html
└ src/
   ├ App.jsx / main.jsx / styles.css
   ├ components/
   │  ├ Layout.jsx / Modal.jsx / Feedback.jsx
   │  ├ TransactionForm.jsx / TransactionList.jsx
   │  └ Charts.jsx / MonthSelect.jsx / EmptyState.jsx
   ├ context/AppContext.jsx
   ├ data/defaults.js
   ├ pages/
   │  ├ DashboardPage.jsx / TransactionsPage.jsx
   │  ├ AnalyticsPage.jsx / BudgetPage.jsx
   │  └ ReportsPage.jsx / SettingsPage.jsx
   └ utils/
      ├ finance.js / validation.js / storage.js
      └ dataIntegrity.js / exportUtils.js

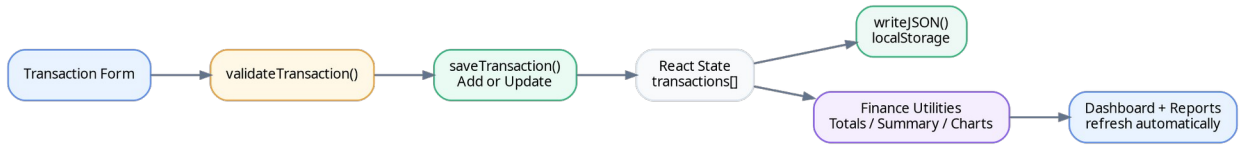
README.md / PROJECT_GUIDE.md / BUILD_AUDIT.md
```

App.jsx	Defines routes, shared transaction modal, confirmation dialog and toast stack.
main.jsx	Mounts React, BrowserRouter and AppProvider; imports global CSS.
AppContext.jsx	Central application state and actions for transactions, budgets, categories, recurring data and settings.
defaults.js	Default categories, payment methods, currencies and demo transactions.
finance.js	Single source of truth for totals, summaries, budget progress, dates and sorting.
dataIntegrity.js	Sanitizes localStorage/backup data and validates data shapes.

SECTION 09

State Management & Data Flow

How a user action becomes persistent data and then updates every dependent screen.



Transaction flow from form input to persistent storage and derived UI.

AppContext is the shared state container. It stores transactions, categories, budgets, recurring templates, settings, selectedMonth and UI feedback state. It also exposes actions such as saveTransaction(), deleteTransaction(), setOverallBudget(), addCategory(), addRecurring(), restoreData() and resetAllData().

> **React Context এখানে কী করছে?**
>
> Context ব্যবহার করার কারণে Dashboard, Transactions, Budget, Reports বা Settings page-কে prop chain দিয়ে একই data বারবার pass করতে হয় না। Provider এক জায়গায় state রাখে এবং useApp() hook দিয়ে child componentগুলো সেই state/action access করে।

Important React hooks used in the project:

useState	Stores mutable UI/application state such as transactions, filters, forms and settings.
useEffect	Persists state to localStorage, applies theme and checks budget thresholds.
useMemo	Avoids recalculating derived lists/totals/charts when source dependencies have not changed.
useCallback	Keeps shared action functions stable and reduces unnecessary function recreation.
useRef	Tracks one-time initialization, duplicate-save guards, alert keys and modal focus management.
useContext	Provides access to the AppContext through the useApp() helper.

SECTION 10

Transaction Data Model & CRUD Logic

The transaction object is the core record from which almost every feature is derived.

Representative transaction shape

```
{
  id: "tx-...",
  type: "expense",
  amount: 850,
  category: "Food",
  title: "Dinner",
  note: "Optional note",
  date: "2026-08-27",
  paymentMethod: "Cash",
  createdAt: "...ISO timestamp...",
  updatedAt: "...ISO timestamp..."
}
```

Add Transaction

The form collects type, amount, category, date, title, note and payment method. TransactionForm validates the input before calling saveTransaction(). AppContext validates again, converts amount to a number, trims strings, generates an ID and inserts the new object into transactions[].

Edit Transaction

Editing passes an existing transaction into the same form. When an id exists, saveTransaction() maps over the existing array and updates the matching record instead of pushing a second transaction. This is why editing does not create duplicates.

Delete Transaction

Deletion is protected by a custom confirmation dialog. Only after confirmation does the state remove the record using filter(). The app then shows a toast and all derived calculations update from the new array.

> **CRUD মানে কী?**
>
> CRUD = Create, Read, Update, Delete. এখানে Create হলো নতুন transaction, Read হলো list/dashboard/chart-এ data দেখা, Update হলো Edit Transaction, আর Delete হলো confirmation-এর পরে transaction remove করা।

SECTION 11

Validation & Data Integrity

Bad input should be rejected before it becomes financial data.

Input validation is centralized in utils/validation.js. The project rejects zero/negative/non-finite amounts, empty titles, invalid transaction types, invalid categories and impossible calendar dates. It also validates budget values and recurring-template frequency/date fields.

Validation rules

```
validateTransaction(input, categories)
- title must be non-empty and <= 80 chars
- amount must be finite and > 0
- type must be "income" or "expense"
- category must exist in the correct category list
- date must be a real YYYY-MM-DD calendar date
```

dataIntegrity.js adds a second defensive layer for persisted and imported data. It sanitizes arrays/objects, normalizes settings, rejects malformed backup schemas, removes invalid records and ensures referenced categories do not disappear from historical data.

> **দুই স্তরের validation কেন?**
>
> Form validation user-কে সঙ্গে সঙ্গে ভুল দেখায়। কিন্তু localStorage বা imported backup manually corrupt হতে পারে। তাই app load/restore-এর সময় dataIntegrity layer আবার data shape check করে। এতে UI valid input-এর ওপর কাজ করে এবং crash হওয়ার risk কমে।

- Protected required category: “Other” cannot be removed in a way that breaks fallback behavior.
- Deleting a used category can reassign historical references instead of leaving invalid transactions.
- Budget values must be finite and non-negative.
- CSV imports are rejected if required columns or valid transaction rows are missing.
- Backup restore validates application identity, schema, settings, transactions and recurring templates.

SECTION 12

Financial Calculation Logic

The key formulas are centralized in utils/finance.js and reused throughout the app.

Current Balance	**All-Time Income - All-Time Expense** calculateBalance(transactions) calls getTotals() across the entire transaction history.
Monthly Savings	**Selected Period Income - Selected Period Expense** calculateMonthlySummary(periodTransactions) returns income, expense and savings together.
Savings Rate	**(Savings / Income) x 100** If income is 0, the function returns null so the UI shows N/A instead of NaN or Infinity.
Category Share	**(Category Expense / Total Expense) x 100** calculateCategoryTotals() groups expense records by category and sorts them by value.
Month-over-Month Change	**((Current - Previous) / Previous) x 100** percentChange() returns null if the previous value is zero to avoid misleading infinite percentages.
Budget Usage	**(Spent / Budget) x 100** calculateBudgetProgress() also returns status, remaining amount and exceeded amount.

Example

ধরা যাক selected month-এ income = ৳60,000 এবং expense = ৳42,000। Savings = 60,000 - 42,000 = ৳18,000। Savings Rate = 18,000 / 60,000 x 100 = 30%। এই value কোনো card-এ manually লেখা নয়; transaction list থেকে dynamically calculate হয়।

Why centralized formulas matter: if the dashboard, report and analytics page each calculated financial values differently, subtle mismatches could appear. Centralized utility functions create one source of truth and make regression testing much easier.

SECTION 13

Monthly, Category & Trend Logic

Raw transactions are transformed into time-based and category-based datasets.

Month Selection

Dates are stored as local calendar keys in YYYY-MM-DD format. monthKey() extracts YYYY-MM and filterByMonth() returns only records for the selected period. “All Time” can be used where a monthly restriction is not appropriate.

Category Aggregation

calculateCategoryTotals() ignores income records, sums expense amounts by category, calculates each category percentage and sorts categories from highest to lowest spending.

Trend Aggregation

groupTransactionsByMonth() creates rows such as {month, income, expense, savings}, sorts by the actual YYYY-MM key and returns the latest requested number of months. This avoids the mistake of sorting month names alphabetically.

> **Date handling গুরুত্বপূর্ণ কেন?**
>
> JavaScript Date এবং UTC conversion ভুলভাবে ব্যবহার করলে Bangladesh timezone-এ date একদিন shift হতে পারে। Final project localDateKey() ব্যবহার করে date-only keys তৈরি করে এবং real calendar date validation করে, যাতে 2026-02-31-এর মতো invalid date reject হয়।

SECTION 14

Budget System & Alerts

Budgeting converts spending totals into actionable limits and status indicators.

The project supports both an overall monthly expense budget and category-specific monthly budgets. Budget values are stored by month, so August and September can have different limits without overwriting each other.

Healthy	0% to below 75% of the configured budget
Approaching Limit	75% to below 100%
Over Budget	100% or more
Remaining	max(Budget - Spent, 0)
Exceeded	max(Spent - Budget, 0)

Budget alerts in AppContext monitor the current month after relevant state changes. A ref-based alert-key set prevents the same threshold warning from being triggered repeatedly on every render.

> **Example**
>
> Food budget = ৳8,000 এবং Food expense = ৳6,800 হলে usage = 85% । App “Approaching Limit” status দেখাতে পারে। Expense ৳9,200 হলে remaining 0 এবং exceeded = ৳1,200 দেখাবে—negative remaining দেখিয়ে confuse করবে না।

SECTION 15

Analytics & Charts

Charts are views of calculated data, not a second source of financial truth.

Recharts receives datasets prepared from transactions. The chart components do not contain hard-coded income or expense values. When transactions change, memoized calculations create new chart data and React updates the visualization.

Income vs Expense Monthly grouped data visualizes cash flow across recent months.	**Expenses by Category** Category totals and percentages show where selected-period money is going.
Monthly Spending Trend Chronological expense values reveal changes over time.	**Top Categories** Ranks expense categories by absolute amount and share.
Largest Expenses Highlights the biggest individual expenses in the selected month.	**Empty States** When there is no data, charts show meaningful empty messaging instead of broken axes.

Chart logic বোঝার shortcut

Chart কখনো localStorage সরাসরি পড়ে না। Flow হলো: localStorage -> AppContext state -> finance utility -> chart data -> Recharts component। এই separation বুঝলে analytics architecture পুরো পরিষ্কার হয়ে যায়।

SECTION 16

Search, Filters & Sorting

Client-side query logic makes transaction history easy to explore.

TransactionsPage holds filter state for search text, type, category, month, payment method, from date, to date and sort order. A useMemo block applies all active conditions together and then calls sortedTransactions() on a copied array.

- Search is case-insensitive and checks title, category, note and payment method.
- Type filter supports All, Income and Expense.
- Category and payment-method filters match exact stored values.
- Month filter compares the YYYY-MM key derived from transaction dates.
- Date range uses inclusive From/To boundaries and rejects From > To.
- Sorting supports newest, oldest, highest amount, lowest amount and category order.
- Clear Filters restores the default filter state.

> **Combined filter example**
>
> যদি Type = Expense, Category = Food, Month = 2026-08 এবং search = “dinner” হয়, তাহলে একটি transaction দেখানোর জন্য চারটি condition-ই true হতে হবে। Filterগুলো আলাদা আলাদা list বানায় না; একই source array-এর ওপর combined predicate চলে।

SECTION 17

Recurring Transaction Templates

A frontend-only implementation that stays honest about browser limitations.

Recurring templates store a future transaction pattern: type, amount, category, title, frequency, next due date and optional payment method. Supported frequencies are weekly, monthly and yearly.

When the application is open, the dashboard checks whether nextDueDate is today or earlier. If a template is due, the user can choose Add, Skip or Later.

Add	Creates a real transaction and advances the template to the next due date.
Skip	Does not create a transaction; only advances the next due date.
Later	Defers the prompt for the current browser session without pretending it was processed.

> **কেন background auto-add নেই?**
>
> Browser tab বন্ধ থাকলে এই frontend app কোনো server process চালাতে পারে না। তাই "Rent automatically added while the browser was closed" claim করা ভুল হতো। App open হলে due template detect করে user-কে decision নিতে দেয়—এটাই technically correct frontend-only behavior।

Recurring date advancement also handles month-end cases. For example, January 31 can safely advance to February 28 rather than becoming an invalid date, and leap-day yearly advancement is normalized.

SECTION 18

CSV, JSON Backup & Restore

Portable data features make local-only storage safer and more useful.

CSV Export

exportCSV() creates Date, Type, Title, Category, Amount, Payment Method and Note columns. Every cell is quoted, internal quotes are doubled, and a UTF-8 BOM is included so Unicode/Bangla text works more reliably in spreadsheet applications.

CSV Import

parseCSV() implements quoted-field parsing and verifies required columns. Imported rows are then sanitized as valid transactions. Corrupted imports are rejected rather than silently inserting partial invalid data.

JSON Backup

The backup represents the complete application state: transactions, categories, budgets, recurring templates and settings. JSON is therefore more complete than CSV, which is mainly transaction portability.

Restore

Restore requires a destructive-replace confirmation and validateBackupPayload() checks the schema before applying data. Wrong-app backups, unsupported/incomplete settings or invalid records are rejected.

> **CSV আর JSON backup-এর difference**
>
> CSV হলো transaction table export—Excel/Sheets-এ খুলতে সুবিধা। JSON backup হলো app-এর full state snapshot—budget, category, recurring, settingsসহ। তাই recovery-এর জন্য JSON বেশি complete।

SECTION 19

localStorage Persistence

How the application keeps data after refresh without a backend.

Persistent browser keys

```
expenseTracker.transactions
expenseTracker.categories
expenseTracker.budgets
expenseTracker.recurring
expenseTracker.settings
expenseTracker.theme
expenseTracker.dataVersion
```

On startup, loadInitialData() reads stored JSON using safe wrappers, sanitizes data shapes and creates the initial Context state. Later, useEffect hooks write updated state back to localStorage whenever transactions, categories, budgets, recurring templates or settings change.

DATA_VERSION is currently 1. Versioning creates a path for future migration logic instead of assuming that stored data will always use the same schema.

> **localStorage কী?**
>
> localStorage হলো browser-এর key-value storage। একই browser/profile-এ page refresh বা browser restart হলেও data থাকে। কিন্তু browser storage clear করলে data চলে যাবে, অন্য device-এ automatically যাবে না, এবং highly sensitive financial information রাখার জন্য এটাকে production database হিসেবে ধরা উচিত নয়।

- safeParse() catches invalid JSON and falls back instead of crashing.
- readJSON()/readString() handle localStorage access failures.
- writeJSON() returns safely if browser storage is unavailable.
- Data-integrity sanitizers ensure malformed shapes do not become trusted application state.

SECTION 20

Reports & Printing

The Reports page converts selected-month data into an A4-friendly summary.

The monthly report reuses the same financial utilities as the dashboard, which keeps totals consistent. It includes the selected period, generated date, currency, income, expenses, savings, savings rate, budget status, top category, transaction count, largest expense and a transaction table.

CSS @media print rules hide navigation, editing controls, theme buttons and unnecessary interactive UI. The user can then use the browser print dialog and choose “Save as PDF” without any external PDF API.

> **Report কেন useful?**
>
> Portfolio demo-তে report feature দেখায় যে project শুধু screen dashboard না; same financial data থেকে printable output generate করতে পারে। এতে CSS print media, table readability এবং cross-feature calculation consistency demonstrate হয়।

SECTION 21

Theme, Currency & Responsive UI

Presentation settings change the experience without changing the financial data.

Theme System

The selected light/dark theme is stored in settings and mirrored to a dedicated theme key. The root document receives a data-theme attribute, while CSS variables control backgrounds, surfaces, text, borders and status colors.

Currency Display

The application supports BDT, USD, EUR, GBP and INR display formatting. formatCurrency() uses currency metadata for symbol and locale. This is formatting only: changing BDT to USD does not convert numeric transaction values using an exchange rate.

Responsive Design

Desktop uses multi-column grids, while tablet layouts adapt column counts. Mobile prioritizes key summary cards, touch-friendly actions, stacked transaction cards and a mobile-friendly modal rather than simply shrinking the desktop table.

> **Currency switch-এর meaning**
>
> Settings-এ BDT থেকে USD করলে 1000 সংখ্যাটা exchange rate অনুযায়ী বদলাবে না; শুধু symbol/format বদলাবে। অর্থাৎ ৳1,000 -> $1,000 display হতে পারে। এটি intentional, কারণ project live currency conversion API ব্যবহার করে না।

SECTION 22

Accessibility & UX Engineering

Small interaction details make the app more usable and more professional.

- Semantic labels and meaningful form error messages.
- Icon-only buttons use accessible names where needed.
- Custom modals and confirmation dialogs support Escape, focus trapping and focus restoration.
- Income and expense states are communicated using text/icons, not color alone.
- Light-theme status colors were adjusted for better contrast.
- Mobile transaction layouts avoid awkward wide-table horizontal scrolling.
- Custom toast notifications replace disruptive browser alert() for normal feedback.
- Dangerous actions such as delete, reset and restore use explicit confirmation flows.

> **Accessibility কেন portfolio value বাড়ায়?**
>
> শুধু সুন্দর UI বানানো frontend skill-এর শেষ নয়। Keyboard user, readable contrast, focus management এবং clear errors consider করা হলে project engineering maturity দেখায়।

SECTION 23

How the Project Was Built

A practical workflow from idea to final audited application.



A simplified development workflow for the project.

1. Define the user problem and keep the scope frontend-only.
2. Design the transaction object and the persistent data groups: transactions, categories, budgets, recurring templates and settings.
3. Create centralized finance/date utilities before spreading calculations across UI components.
4. Create the AppContext state/action layer so all pages share the same source of truth.
5. Build the common shell, navigation, modal, feedback components and transaction form.
6. Implement Dashboard and Transactions first because they validate the core data model.
7. Add budgets, analytics, recurring templates, reports and settings as derived features.
8. Add localStorage persistence, CSV/JSON portability and schema sanitization.
9. Polish dark/light themes, mobile layouts, print CSS and accessibility behaviors.
10. Audit edge cases such as zero income, invalid dates, previous-month zero, corrupted storage and over-budget values.

> **Idea থেকে implementation-এর আসল lesson**
>
> আগে UI আঁকা আর পরে logic বসানো সবসময় best approach না। এই ধরনের app-এ আগে data model ও formula পরিষ্কার করলে UI componentগুলো predictable হয়। Financial app-এর ক্ষেত্রে "data first, derived logic second, presentation third" চিন্তা করা বেশি reliable।

SECTION 24

How to Run & Build the Project

Exact commands for VS Code, local development and production verification.

Recommended local folder: F:\Projects\expense-tracker-localstorage

Prerequisites: install a current Node.js LTS version and npm. Then open the project folder in VS Code or a terminal.

Development

```
cd F:\Projects\expense-tracker-localstorage
npm install
npm run dev
```

Vite will print a local URL, commonly http://localhost:5173. Open that address in the browser.

Production verification

```
npm run build
npm run preview
```

npm run build creates an optimized production bundle (normally in dist/). npm run preview serves that production build locally for final checking.

npm install	Reads package.json and installs React, Vite, Router, Recharts, Lucide and related dependencies.
npm run dev	Starts the Vite development server with fast reload.
npm run build	Creates the production bundle and catches bundling/import issues.
npm run preview	Serves the built output locally for production-style testing.

> **Important local check**
>
> GitHub-এ publish করার আগে তোমার Windows PC-তে npm install এবং npm run build একবার successful হওয়া confirm করো। node_modules GitHub-এ push করার দরকার নেই; package.json থেকেই অন্য machine dependency install করতে পারবে।

SECTION 25

Testing & Final Audit

What was checked, what passed, and what still requires local verification.

The final project audit covered source integrity, financial formulas, date handling, storage/backup behavior and CSV edge cases.

JS/JSX Source Audit	PASS - 23 source files passed transpile syntax checks.
Relative Import Audit	PASS - local source imports resolved to existing files.
Financial Regression Tests	PASS - balance, income/expense, savings, savings rate, category totals, budget states and sorting.
Date Regression Tests	PASS - leap dates, invalid dates, month-end recurring and yearly leap-day advancement.
Storage / Backup Tests	PASS - malformed data fallback, invalid backup rejection and safe storage failures.
CSV Tests	PASS - commas, escaped quotes, Unicode/Bangla, required-column rejection and unterminated quote rejection.
Production Vite Build in restricted generation sandbox	BLOCKED - npm registry DNS EAI_AGAIN prevented dependency download, so Vite was unavailable there.

Accuracy note

Restricted generation environment-এ npm registry resolve না হওয়ায় full Vite build pass claim করা হয়নি। এটা source syntax failure পাওয়া গেছে—এমন নয়। তোমার normal internet-connected PC-তে npm install && npm run build চালিয়ে production build verify করাই final local step।

Notable fixes from the final audit included: month filtering, date-range validation, strict calendar dates, corrupted localStorage sanitization, category consistency, over-budget “exceeded” values, complete category chart totals, recurring Later behavior, duplicate demo-data protection, stronger backup validation, focus handling and improved status contrast.

SECTION 26

What I Learned

Skills and concepts this project helps demonstrate in a portfolio or viva.

এই **project** থেকে আমার সবচেয়ে গুরুত্বপূর্ণ শেখাগুলো:

- React component-based architecture ব্যবহার করে বড় UI-কে ছোট reusable অংশে ভাঙা।
- Context API দিয়ে shared state এবং actions centralize করা।
- useMemo দিয়ে derived calculations ও filtered lists optimize করা, কিন্তু unnecessary over-optimization না করা।
- Business logic UI-এর ভেতরে ছড়িয়ে না দিয়ে utility function-এ রাখা।
- Income, expense, balance, savings, savings rate, budget progress এবং category percentage mathematically handle করা।
- Zero denominator, invalid amount, invalid date এবং malformed JSON-এর মতো edge case handle করা।
- localStorage কীভাবে persistence দেয় এবং এর security/synchronization limitation কী।
- CSV quoting, Unicode handling, Blob API এবং client-side download process বোঝা।
- JSON backup/restore-এর সময় schema validation কেন দরকার।
- Recharts-কে actual derived data দিয়ে chart render করানো।
- Responsive design-এ desktop UI শুধু shrink না করে mobile interaction আলাদাভাবে ভাবা।
- Modal focus trap, confirmation, toast, contrast এবং semantic form handling-এর accessibility value বোঝা।
- A4 print CSS দিয়ে browser থেকেই report-to-PDF workflow তৈরি করা।
- Final audit-এর সময় source, formula, data integrity এবং honest limitation documentation কত গুরুত্বপূর্ণ।

> **Career takeaway**
>
> এটি শুধু "React জানি" বলার project নয়। এখানে state management, data modeling, business logic, persistence, charts, import/export, defensive programming, responsive design এবং testing mindset একসাথে demonstrate করা যায়।

SECTION 27

Limitations & Privacy

A good project description explains what the software cannot safely do.

- Data exists only in the current browser/profile unless the user exports a backup.
- Clearing browser storage can remove local data.
- No automatic synchronization between phone, laptop or multiple browsers.
- No secure user-account system or server-side access control.
- localStorage is not appropriate for highly sensitive production financial data.
- Recurring templates cannot run while the browser is closed.
- Currency switching changes display formatting only and does not perform exchange-rate conversion.
- No bank API or automated transaction import from financial institutions.

Privacy message

“Your data stays in this browser. This demo does not send transaction data to a server.” — এই claim project-এর frontend-only architecture-এর সাথে consistent। তবে local device access থাকলে localStorage data theoretically inspect করা সম্ভব, তাই এটাকে encrypted banking vault বলা যাবে না।

SECTION 28

Future Improvements

How the project could evolve from a portfolio demo into a larger system.

Backend API Move persistence to a server using Node/Express or another backend framework.	**Database** Use PostgreSQL/MySQL for durable multi-user data and indexed queries.
Authentication User accounts, sessions, password security and authorization boundaries.	**Cloud Sync** Access the same data across devices with secure server persistence.
PWA Installable offline shell, service-worker caching and richer offline behavior.	**Testing Suite** Automated unit/component/end-to-end tests with CI on every GitHub push.
Advanced Import Bank-statement mapping and configurable column import with user confirmation.	**Security Upgrade** Server-side validation, encryption strategy, secrets management and audit logging.

Future backend version-এর mental model

বর্তমান localStorage layer-কে API calls দিয়ে replace করা যাবে। UI এবং finance utility-এর বড় অংশ একই থাকতে পারে; persistence/action layer server-এর সাথে communicate করবে। এই separation-ই current architecture-এর future value।

SECTION 29

Portfolio Presentation Notes

How to explain the project in a concise, credible way.

Suggested Portfolio Summary

> **One-line pitch**
>
> A frontend-only personal finance dashboard built with React and Vite that tracks transactions, budgets and analytics locally in the browser, with CSV/JSON portability and printable monthly reports.

Best points to highlight in an interview or project card:

- Centralized financial calculation utilities instead of duplicated formulas.
- Versioned/sanitized localStorage rather than blindly trusting stored JSON.
- Budget and analytics features generated from real transaction data.
- Honest recurring-template behavior for a browser-only architecture.
- CSV export/import plus full JSON backup/restore.
- Responsive light/dark UI and print-ready reports.
- Final regression audit covering financial, date, storage and CSV edge cases.

What not to claim: production banking security, server authentication, automatic cloud backup, live currency conversion or background recurring execution. Being precise about limitations makes the project more credible, not less.

SECTION 30

Viva Questions & Answers

Short, accurate answers based on the actual final implementation.

1. What is the main purpose of this project?

To track personal income and expenses and derive balance, monthly summaries, budgets, analytics and reports entirely in the browser.

2. Why did you use React?

React makes the UI component-based and automatically re-renders dependent views when shared transaction state changes.

3. Why did you use Vite?

Vite provides the development server, modern module handling and production bundling with a simple React setup.

4. What is localStorage?

A browser key-value storage API that persists strings across refreshes in the same browser/profile.

5. Why is no database required?

The intended scope is a local personal demo; JSON state can be stored in localStorage without multi-user or cloud requirements.

6. How is current balance calculated?

All-time income minus all-time expense using calculateBalance(transactions).

7. How is monthly savings calculated?

Selected-period income minus selected-period expense.

8. What happens when income is zero?

Savings rate returns null and the UI displays N/A instead of Infinity or NaN.

9. How is category percentage calculated?

Category expense divided by total selected-period expense, multiplied by 100.

10. How do month-over-month comparisons avoid Infinity?

percentChange() returns null when the previous value is zero.

11. Where are financial formulas stored?

Primarily in src/utils/finance.js so all pages can reuse the same logic.

12. What does AppContext do?

It stores shared state and exposes actions for CRUD, budgets, categories, recurring templates, backup/restore and settings.

13. What is useMemo used for?

It memoizes derived values such as filtered transactions, summaries and chart datasets until dependencies change.

14. How does edit avoid creating a duplicate?

If an existing transaction id is present, saveTransaction maps and replaces that record instead of inserting a new one.

15. How are invalid transactions prevented?

Form validation and AppContext validation reject invalid data; dataIntegrity also sanitizes persisted/imported records.

16. How are dates stored?

As local YYYY-MM-DD calendar keys for transaction/due dates, avoiding unnecessary UTC conversion for date-only values.

17. What is the budget status logic?

Below 75% is Healthy, 75-99% is Approaching Limit and 100%+ is Over Budget.

18. How do charts get data?

Finance utilities aggregate the transaction state and Recharts renders those derived arrays.

19. Are chart values hard-coded?

No. They are generated from the current transaction data.

20. How does CSV export handle commas and quotes?

Every cell is quoted and internal double quotes are escaped by doubling them.

21. Why include JSON backup if CSV exists?

JSON preserves the full app state including categories, budgets, recurring templates and settings; CSV mainly represents transactions.

22. How does recurring automation work?

The app checks due templates when it is open and offers Add, Skip or Later; it does not run in the background while closed.

23. How is dark mode implemented?

Settings apply a theme value to the document root and CSS variables change the visual tokens.

24. Does changing currency convert money?

No. It only changes display symbol/locale; transaction numbers stay the same.

25. What are the major limitations?

No backend, no account security, no cross-device sync, localStorage can be cleared, and it is not suitable for highly sensitive production data.

26. Why is this a strong portfolio project?

It combines UI engineering with business logic, persistence, data integrity, analytics, import/export, reporting, responsive design and accessibility.

SECTION 31

Quick Revision Cheat Sheet

One-page mental model before a demo or viva.

Core Data	transactions[] is the main financial record set.
State	AppContext + React hooks.
Persistence	localStorage JSON with safe parsing and dataVersion = 1.
Balance	all-time income - all-time expense.
Savings	period income - period expense.
Savings Rate	savings / income x 100; N/A when income = 0.
Budget	spent / budget x 100; Healthy <75, Approaching 75-99, Over >=100.
Analytics	finance.js prepares data; Recharts renders it.
Search/Filter	Client-side combined predicates + copied-array sorting.
Recurring	Due check while app is open -> Add / Skip / Later.
CSV	Transactions only; quoted UTF-8 export/import.
JSON Backup	Full state snapshot with validation before restore.
Reports	Same calculations reused; @media print for A4 output.
Theme	CSS variables driven by data-theme.
Currency	Formatting only, no exchange-rate conversion.
Run	npm install -> npm run dev.
Build	npm run build -> npm run preview.
Main Limitation	Browser-local data; no backend/cloud account security.

30-second explanation

“Expense Tracker is a React + Vite frontend-only finance dashboard. Transactions are stored in browser localStorage. AppContext manages shared state, while finance utilities calculate balance, monthly summaries, category totals and budgets. Recharts visualizes derived data. The app also supports filters, recurring templates, CSV/JSON portability, printable reports, theme/currency settings and responsive UI. It intentionally has no backend, so data stays on the current browser.”

SECTION END

Closing Note

A project is strongest when you can explain the decisions behind it, not only show the interface.

The final Expense Tracker demonstrates how a relatively familiar idea can become a serious portfolio project when the implementation includes clear data modeling, deterministic financial logic, defensive persistence, useful analytics, accessible interactions and honest architectural limits.

> নিজের **demo-**এর আগে কী করবে
>
> Project local folder-এ npm install, npm run build এবং npm run dev চালিয়ে verify করো। তারপর Demo Data load করে Dashboard -> Transactions -> Budget -> Analytics -> Reports -> Settings এই sequence-এ একবার walkthrough practice করো। Formula এবং localStorage limitation নিজের ভাষায় বলতে পারলেই viva/interview-এ project confidently explain করা সহজ হবে।

EXPENSE TRACKER

Project Documentation

Created by Ann Noor Muhammad Mrinmoy